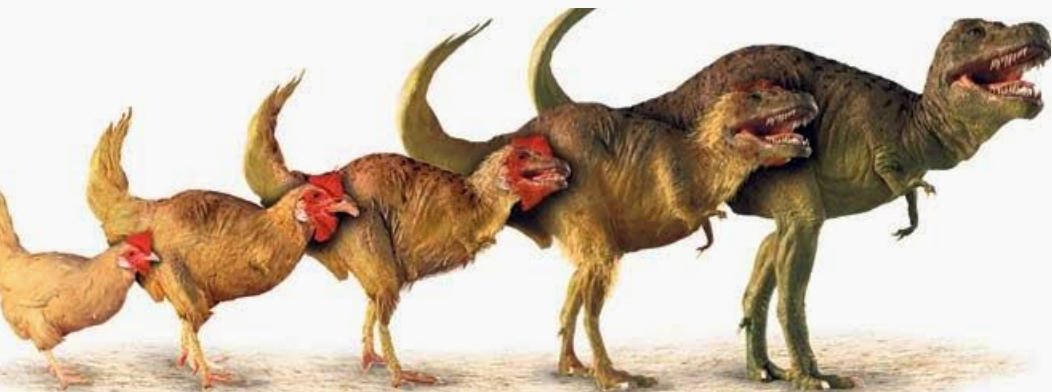


Evolutionary coding agent: OpenEvolve

Sânziana Stelea (sstelea@ethz.ch),

Christina Tsakanika (CX)



Mathematical discovery at scale



Today, we share a breakthrough on the planar unit distance problem, a famous open question first posed by Paul Erdős in 1946.



For the first time ever, we show how a large language model can be used to discover novel solutions to long-standing problems in math and computer science.



Yossi Matias • 2nd

Vice President, Google. Head of Google Research.

10mo • Edited •

[+ Follow](#) ...

A new research demonstrates how an LLM-powered coding agent can help discover new mathematical structures that push the boundaries of our understanding of complexity theory.



hello there the jacobian conjecture is false thanx to my close friend akhil for asking about it and my other close friend fable for working during the world cup final

$((1+xy)^3 z + y^2 (1+xy) (4+3xy), y + 3x (1+xy)^2 z + 3xy^2 (4+3xy), 2x - 3x^2 y - x^3 z): \mathbb{C}^3 \rightarrow \mathbb{C}^3$, has jacobian determinant -2 , and sends $(0, 0, -1/4)$, $(1, -3/2, 13/2)$, and $(-1, 3/2, 13/2)$ to $(-1/4, 0, 0)$

5:19 AM · Jul 20, 2026 · **39.8M** Views

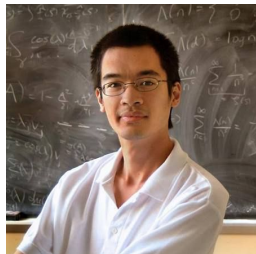
Mathematical discovery at scale

Google DeepMind

AlphaEvolve: A coding agent for scientific and algorithmic discovery

Alexander Novikov*, Ngán Vũ*, Marvin Eisenberger*, Emilien Dupont*, Po-Sen Huang*, Adam Zsolt Wagner*, Sergey Shirobokov*, Borislav Kozlovskii*, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli and Matej Balog*
Google DeepMind¹

Terence Tao



- Fields medal 2006
- Youngest appointed Prof. at UCLA

MATHEMATICAL EXPLORATION AND DISCOVERY AT SCALE

BOGDAN GEORGIEV, JAVIER GÓMEZ-SERRANO, TERENCE TAO, AND ADAM ZSOLT WAGNER

Roadmap



01 The idea

Search, but the mutation operator has read the literature.



02 The machine

Alpha/ OpenEvolve's four components and how they interact



03 The evidence

Mathematics, infrastructure, industry



04 The catch

Where it fails, and when it's worth using

01 The idea

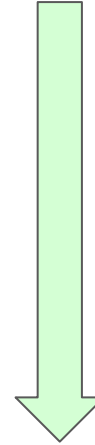


Automated search

Search problem



Initial naive solution



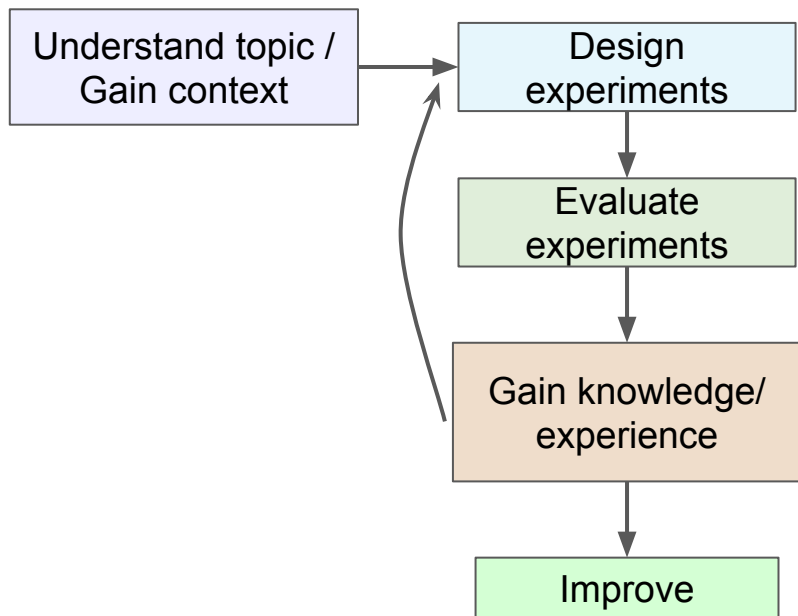
Optimal solution



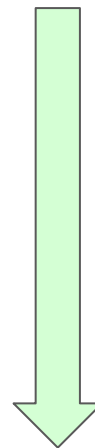
Automated search

Search problem

Human optimization loop



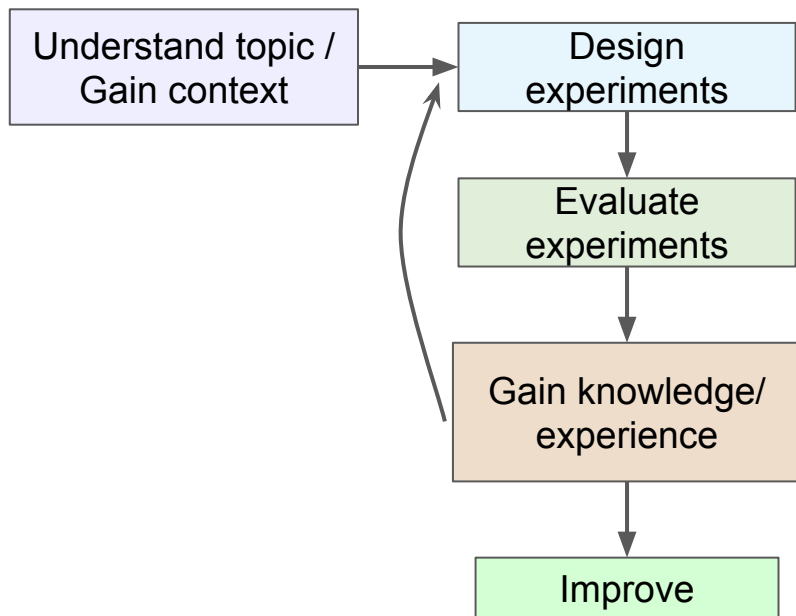
Initial naive solution



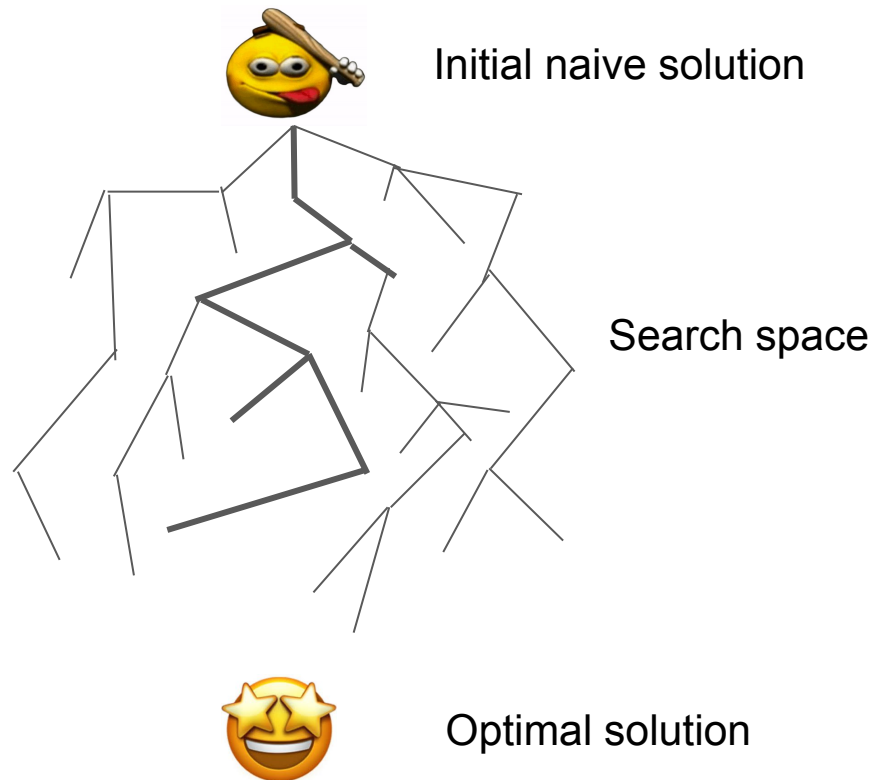
Optimal solution

Automated search

Human optimization loop

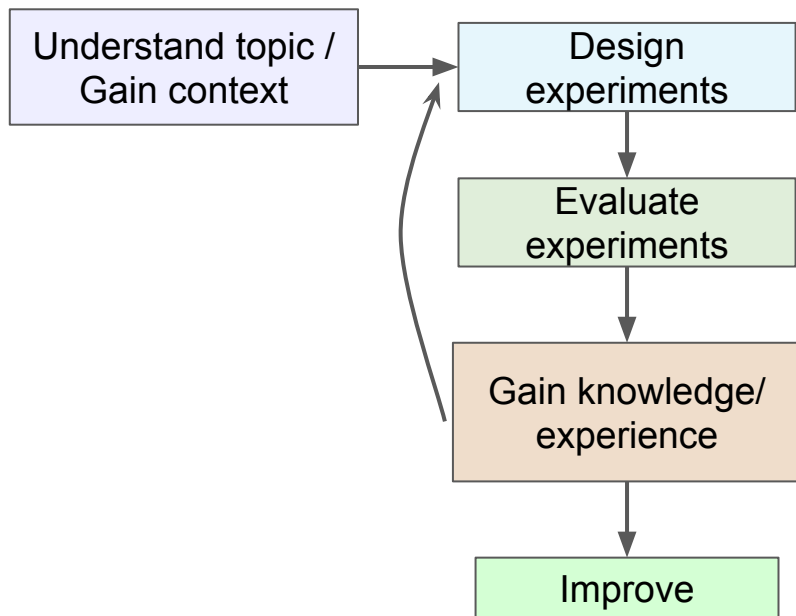


Search problem

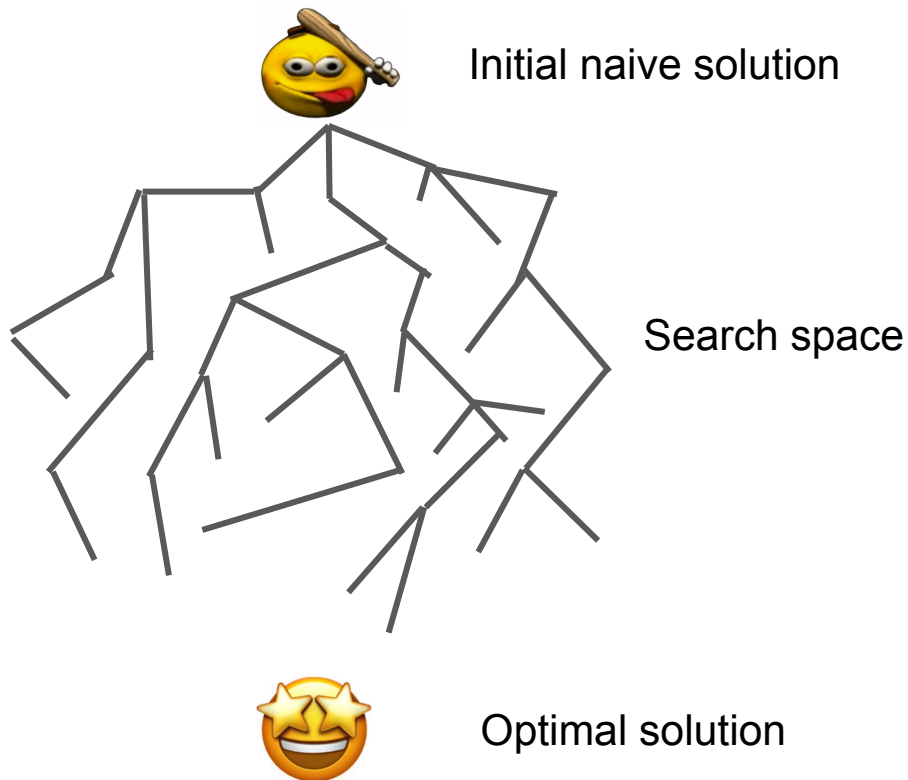


Automated search

Automated optimization loop

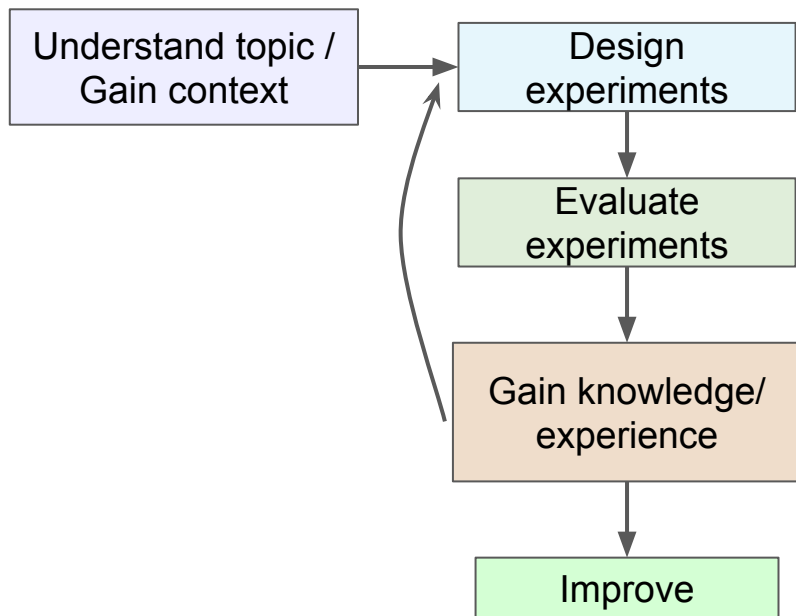


Search problem

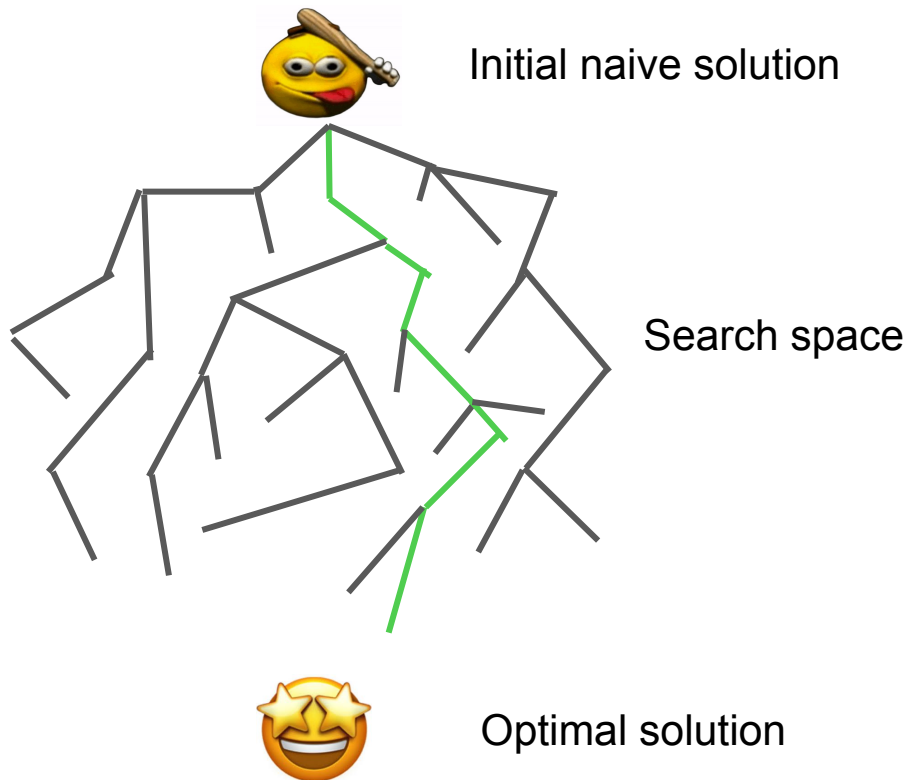


Automated search

Automated optimization loop

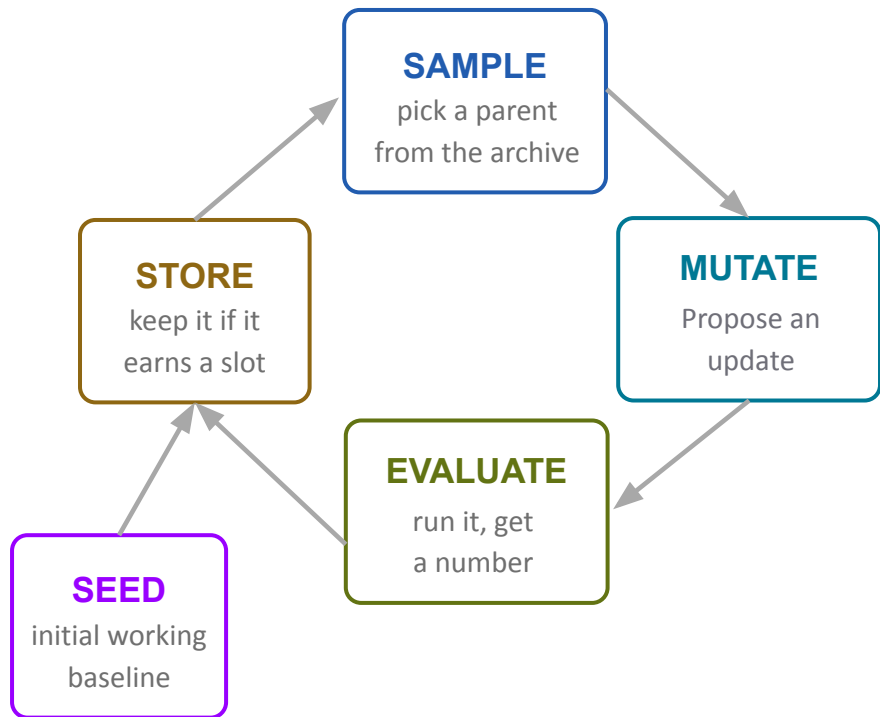


Search problem



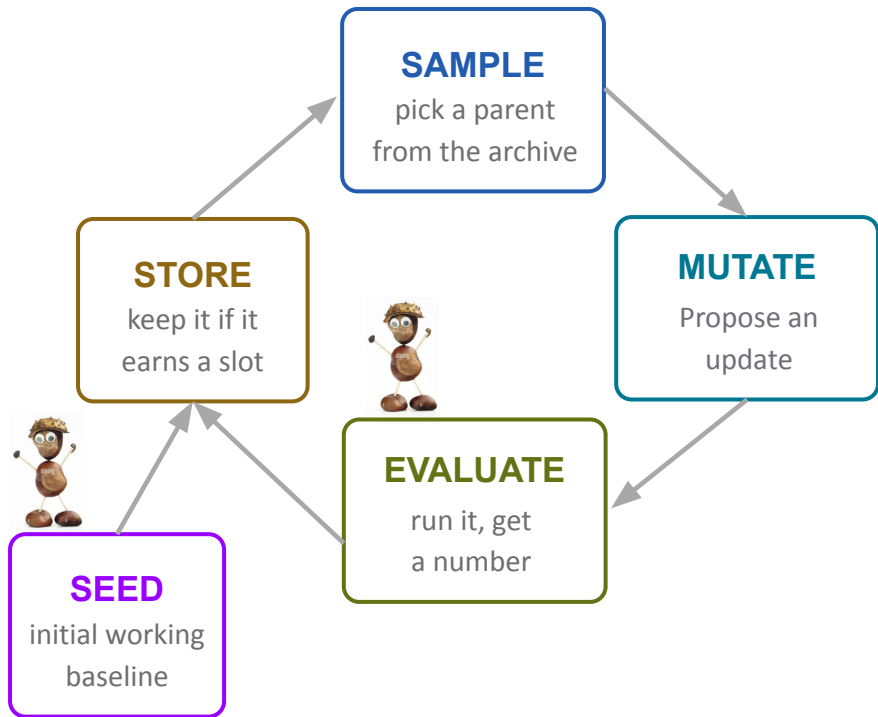


Evolutionary optimization loop





Evolutionary optimization loop



The two things a human supplies

A seed program

Any working baseline, however bad.

A scoring function

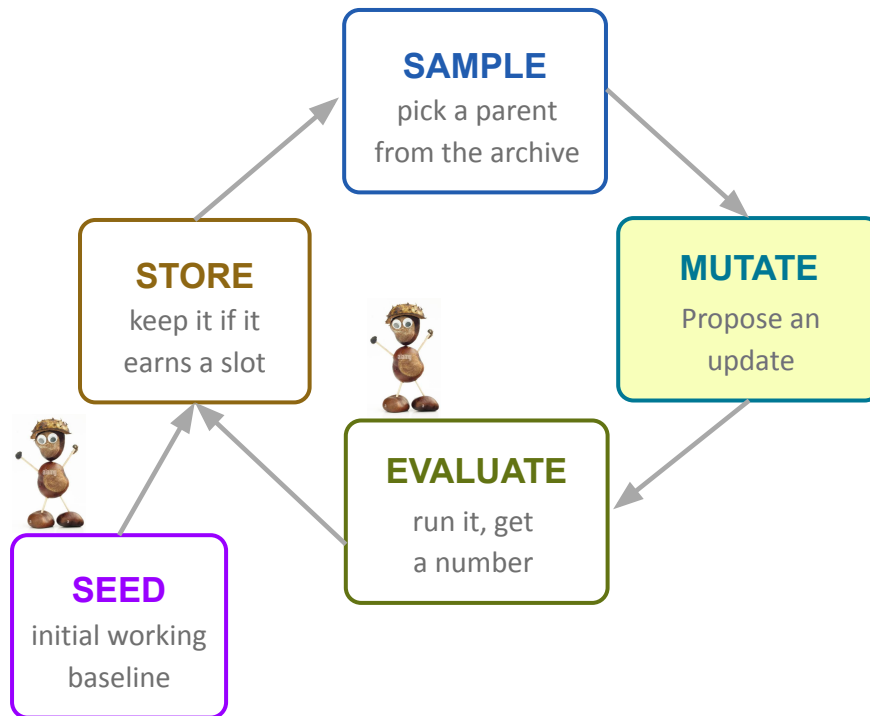
Automatic, deterministic-ish, and honest.

The agent supplies the "how".

You keep ownership of the "what".



Evolutionary optimization loop



1992

Automatic Programming of Robots using Genetic Programming

John R. Koza

Stanford University Computer Science Department
Stanford, CA 94305 USA

E-MAIL: Koza@Sunburn.Stanford.Edu

PHONE: 415-941-0336 FAX: 415-941-9430

James P. Rice

Stanford University Knowledge Systems Laboratory
701 Welch Road

Palo Alto, CA 94304 USA

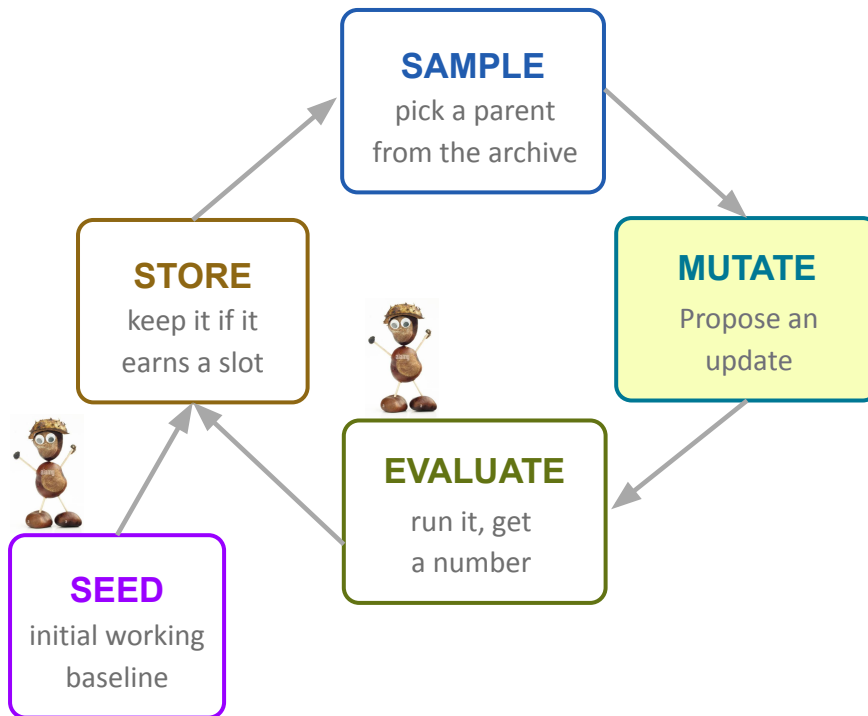
Rice@Sumex-Aim.Stanford.Edu 415-723-8405

Classical mutation

- Perturb a syntax tree at random
- **Valid code, but semantically blind**



Evolutionary optimization loop



1992

Automatic Programming of Robots using Genetic Programming

John R. Koza

Stanford University Computer Science Department
Stanford, CA 94305 USA

E-MAIL: Koza@Sunburn.Stanford.Edu
PHONE: 415-941-0336 FAX: 415-941-9430

James P. Rice

Stanford University Knowledge Systems Laboratory
701 Welch Road

Palo Alto, CA 94304 USA
Rice@Sumex-Aim.Stanford.Edu 415-723-8405

Classical mutation

- Perturb a syntax tree at random
- **Valid code, but semantically blind**

LLM mutation

- LLM proposes the mutation
- **Edits are plausible** given the code's meaning and the whole scientific literature



A three-year lineage

2023–24 FunSearch

Nature. Evolves a single Python function. Proof of concept that an LLM can produce new, provably correct mathematical knowledge.

2025 AlphaEvolve

Whole files instead of functions. Diff-based edits, an LLM ensemble (fast + strong), multiple metrics, evolved meta-prompts. Closed source.

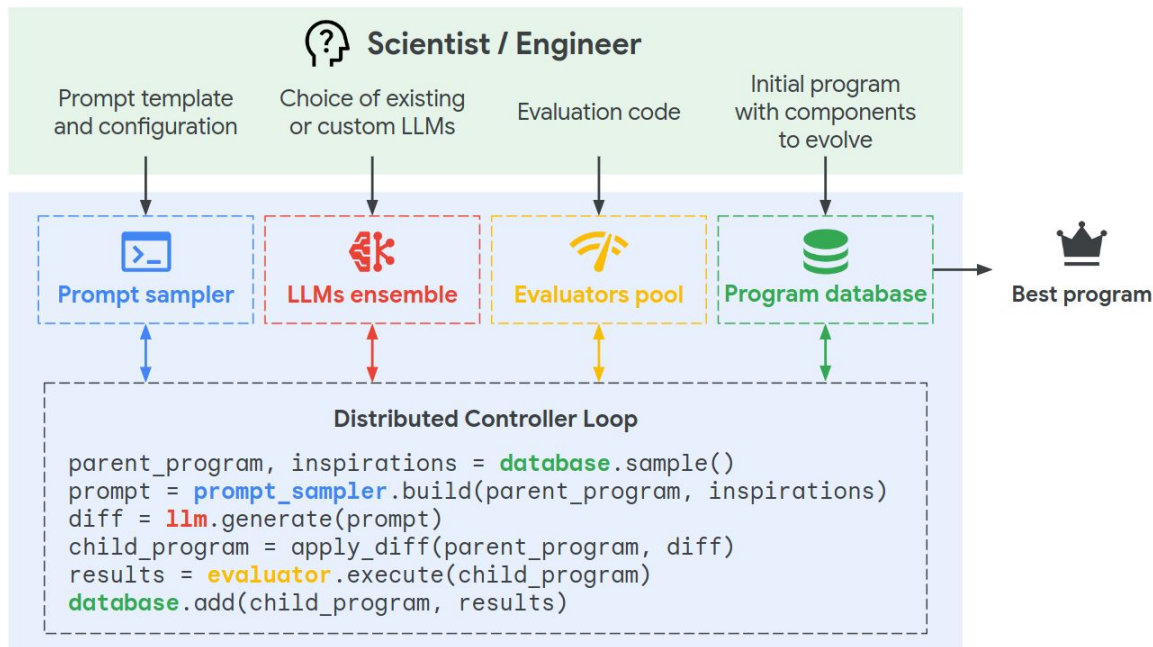
2025–26 The open ecosystem

OpenEvolve, ShinkaEvolve, CodeEvolve, GigaEvo, ThetaEvolve, DeepEvolve

02 The machine



Six lines of pseudocode, four moving parts



■ Prompt sampler

Builds the context: parent, rivals, scores, errors.

■ LLM ensemble

Cheap+fast for breadth, strong+slow for depth.

■ Evaluators pool

Your code. The only source of truth.

■ Program database

Decides who gets to reproduce.

Asynchronous by design: the bottleneck is evaluator throughput, not the LLM.



The prompt is where the search actually happens

■ Parent

- The program to be mutated, in full.

■ Inspirations

- A few top performers
 - A few deliberately diverse ones, with their scores attached
- ⇒ The model sees a landscape, not a single example.

■ Artifacts

- Failed assertions can be fed back. The model debugs its own previous attempt.

■ Meta-prompt

- The instruction template itself is an evolvable object
(See lecture on GEPA)

```
# EVOLVE-BLOCK START
"""Image classification experiment in jaxline."""

import jax
...
# EVOLVE-BLOCK-END

...

# EVOLVE-BLOCK-START
class ConvNet(hk.Module):
    def __init__(self, num_classes): ...
    def __call__(self, inputs, is_training): ...

def sweep():
    return hyper.zipit[...]
# EVOLVE-BLOCK-END

...

def evaluate(eval_inputs) -> dict[str, float]:
    ...
    return metrics
```

(a) 
User-provided file

Act as an expert software developer. Your task is to iteratively improve the provided codebase. [...]

- Prior programs

Previously we found that the following programs performed well on the task at hand:

top_1_acc: 0.796; neg_eval_log_loss: 0.230; average_score: 0.513

```
"""Image classification experiment in jaxline."""
[...]
```

```
class ConvNet(hk.Module):
    """Network."""

    def __init__(self, num_channels=32, num_output_classes=10):
        super().__init__()
        self.conv1 = hk.Conv2D(num_channels, kernel_shape=3)
        self.conv2 = hk.Conv2D(num_channels * 2, kernel_shape=3)
        self.conv3 = hk.Conv2D(num_channels * 4, kernel_shape=3)
        self.logits_module = hk.Linear(num_output_classes)

[...]
```

(b) 

Figure: Novikov et al., AlphaEvolve

The current model uses a simple ResNet architecture with only three ResNet blocks. We can improve its performance by increasing the model capacity and adding regularization. This will allow the model to learn more complex features and generalize better to unseen data. We also add weight decay to the optimizer to further regularize the model and prevent overfitting. AdamW is generally a better choice than Adam, especially with weight decay.

```
<<<<<< SEARCH
self._block1 = ResNetBlock(num_channels)
self._block2 = ResNetBlock(num_channels * 2, stride=2)
self._block3 = ResNetBlock(num_channels * 4, stride=2)
=====
self._block1 = ResNetBlock(num_channels)
self._block2 = ResNetBlock(num_channels, stride=1)
self._block3 = ResNetBlock(num_channels * 2, stride=2)
self._block4 = ResNetBlock(num_channels * 2, stride=1)
self._block5 = ResNetBlock(num_channels * 4, stride=2)
self._block6 = ResNetBlock(num_channels * 4, stride=1)
>>>>>> REPLACE
<<<<<< SEARCH
def optimizer(self, learning_rate):
    return optax.adam(learning_rate)
=====
def optimizer(self, learning_rate):
    return optax.adamw(learning_rate, weight_decay=1e-4)
>>>>>> REPLACE
```

(c) 

Example output
generated by the
LLM

- Current program

Here is the current program we are trying to improve (you will need to propose a modification to it below).

top_1_acc: 0.862; neg_eval_log_loss: 0.387; average_score: 0.624

```
"""Image classification experiment in jaxline."""
[...]
```

```
class ConvNet(hk.Module):
    """Network."""

    def __init__(self, num_channels=32, num_output_classes=10):
        super().__init__()
        self.conv1 = hk.Conv2D(num_channels, kernel_shape=3)
        self._block1 = ResNetBlock(num_channels)
        self._block2 = ResNetBlock(num_channels * 2, stride=2)
        self._block3 = ResNetBlock(num_channels * 4, stride=2)
        self._logits_module = hk.Linear(num_output_classes)

[...]
```

SEARCH/REPLACE block rules:
[...]

Make sure that the changes you propose are consistent with each other. For example, if you refer to a new config variable somewhere, you should also propose a change to add that variable.

Example:
[...]

Task
Suggest a new idea to improve the code that is inspired by your expert knowledge of optimization and machine learning.

Describe each change with a SEARCH/REPLACE block.

Assembled prompt to be provided to the LLMs



The evaluator is the only source of truth

■ The eval function

- A dictionary of scalar metrics, maximised by convention
- Can be three lines returning a graph's size, or a full training run

■ Evaluation cascade

- Test cases of increasing difficulty
- Candidate advances only if it did well at every earlier stage
- Kill faulty programs cheaply

■ LLM-generated feedback

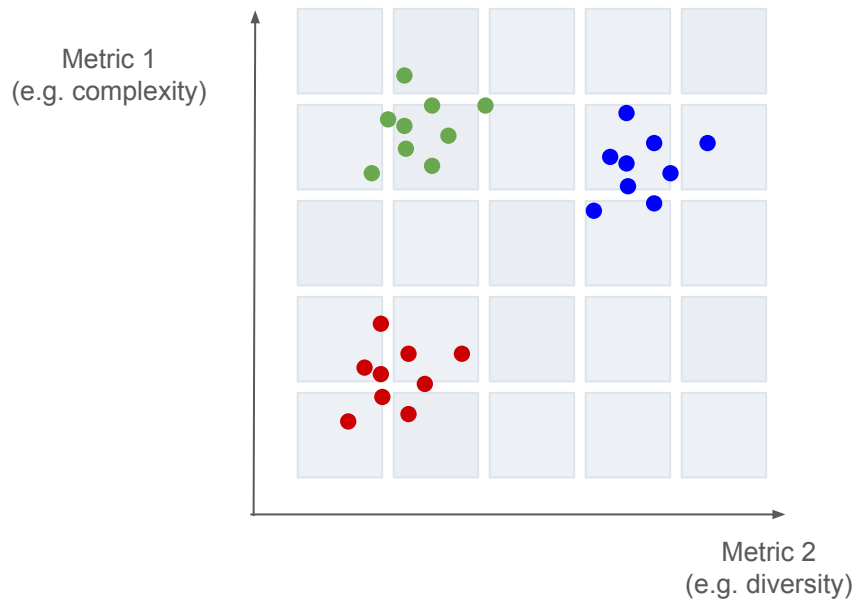
- For properties numbers can't express
- Separate LLM call grades the candidate

■ Eval time

- FunSearch needed under 20 min on one CPU
- AlphaEvolve can spend hours



The database

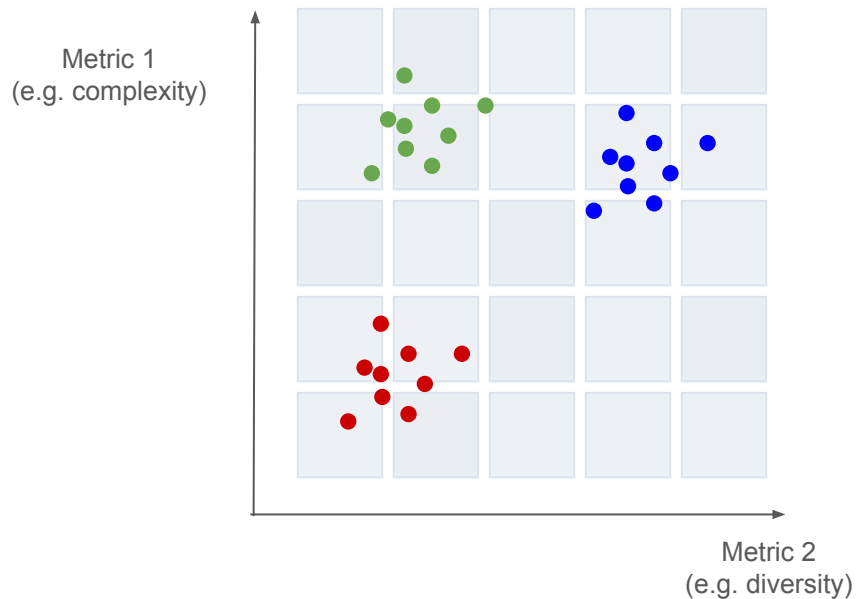


MAP-Elites archive

- ❖ **Bin programs by behaviour**
 - ❖ Score = average over remaining metrics
 - ❖ **Competition is local:** Each cell keeps its own champion
 - ❖ Child program competes with the champion of the cell
- Slow-but-clever program is never deleted by a fast-and-dumb one*



The database



MAP-Elites archive

- ❖ **Bin programs by behaviour**
 - ❖ Score = average over remaining metrics
 - ❖ **Competition is local:** Each cell keeps its own champion
 - ❖ Child program competes with the champion of the cell
- Slow-but-clever program is never deleted by a fast-and-dumb one*

Sampling parent program

- Between island: uniform
- Inside a branch:
 - 70% from the elites (exploitation)
 - 20% pick random (exploration)
 - 10% score-weighted probabilistic selection

03 The evidence

Mathematics



Faster matrix multiplication via finding novel algorithms for tensor decomposition

$$\max_{-1/2 \leq x \leq 1/2} \int_0^1 f(x-x)f(x) dx \geq \mathbb{E} \left(\int_{-1/4}^{1/4} f(x) dx \right)^2$$
 1.5098 $\rightarrow$ 1.5053

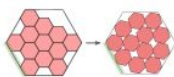
$$\|f * f\|_2^2 \leq \mathbb{E} \|f * f\|_1 \|f * f\|_\infty$$
 0.8892 $\rightarrow$ 0.8962

$$\max_{-1/2 \leq x \leq 1/2} \left| \int_0^1 f(x-x)f(x) dx \right| \geq \mathbb{E} \left(\int_{-1/4}^{1/4} f(x) dx \right)^2$$
 1.4581 $\rightarrow$ 1.4557

$$A(f)A(\hat{f}) \geq \mathbb{E}^{III}$$
 0.3523 $\rightarrow$ 0.3521

Analysis

Hexagon outer edge
 4.000 $\rightarrow$ 3.942



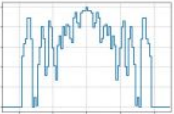
Max distance/min distance
 12.890 $\rightarrow$ 12.889



Sum of radii
 2.6340 $\rightarrow$ 2.6358



Geometry



$$\sup_{x \in [-2,2]} \int_{-1}^1 f(t)g(x+t) dt \geq \mathbb{E}$$
 0.380926 $\rightarrow$ 0.380924

$$|A+B| \ll |A|$$

$$|A-B| \gg |A|$$
 1.1446 $\rightarrow$ 1.1584

Combinatorics

Terence Tao paper

23 / 67 problems where
 SOTA was improved

36 / 67 matched the best
 known construction

Figure 5 | Examples of SOTA-breaking mathematical constructions discovered with *AlphaEvolve*. The versatility of *AlphaEvolve* allows us to tackle problems in analysis (autocorrelation and uncertainty inequalities), geometry (packing and minimum/maximum distance problems) and combinatorics (Erdős's minimum overlap problem and sums and differences of finite sets).



The part that pays for the research

Across Google's stack

0.7%

of Google's fleet-wide compute recovered, continuously, by an evolved Borg scheduling heuristic for stranded resources

23%

average speedup across Gemini's matmul kernels — about 1% of total training time

20%

reduction in Spanner write amplification via evolved LSM-tree compaction heuristics

30%

fewer variant-detection errors in DeepConsensus DNA sequencing correction

14% → 88%

feasible-solution rate for a trained GNN surrogate on AC optimal power flow

10×

lower error on Willow quantum-processor circuits for molecular simulation



And outside Google's walls

FINANCIAL SERVICES

Klarna

2× training speed

Improving one of its largest transformer models

SEMICONDUCTOR MANUFACTURING

Substrate



Multi-fold speedup

of their computational lithography framework

LOGISTICS

FM Logistic



10.4% routing gain

achieving over 15,000 km of annual travel removed

ADVERTISING & MARKETING

WPP



10% accuracy gain

in their AI models navigating campaign data

MATERIALS & LIFE SCIENCES

Schrödinger



~4× faster MLFF

shortening R&D cycles in drug discovery

04 The catch



The slide Google's blog post doesn't have

Simple Baselines are Competitive with Code Evolution

Yonatan Gideoni¹ Sebastian Risi² Yarin Gal¹

¹ Oxford ² Sakana AI & ITU Copenhagen
arXiv:2602.16805 · February 2026

The intellectual work migrated. It did not disappear — it moved from designing the algorithm to designing the search space and the score.

- 01 Google publishes neither the compute budget behind its results nor the prompts. AlphaEvolve is closed, so nobody outside can reproduce it, cost it, or run it against a baseline.
- 02 **Designing the search space (evaluation) and the domain knowledge in the prompt are what set the performance ceiling. The pipeline itself is secondary.**
- 03 Reformulating one problem's search space improved the bound more than any pipeline difference did, for every method at once. The reformulation was done by a human.



Is your problem shaped like this?

1 Can you write down a search space?

Something a program can output.

This is the step that actually needs your expertise.

2 Can a candidate be scored automatically?

Seconds to minutes, no human in the loop, no manual judgement.

3 Is the score honest?

Is there a cheap degenerate solution that scores well?

Assume it will be found.

4 Is a few percent worth it?

These systems deliver small margins reliably. At scale that is money

5 Do you have a baseline and a held-out set?

Otherwise you cannot tell improvement from overfitting to your own evaluator.

References

Novikov et al. (2025)

AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131

Georgiev, Gómez-Serrano, Tao & Wagner (2025)

Mathematical exploration and discovery at scale. arXiv:2511.02864 — plus the problem repository on GitHub

Romera-Paredes et al. (2024)

Mathematical discoveries from program search with large language models. Nature 625 (FunSearch)

Gideoni, Risi & Gal (2026)

Simple Baselines are Competitive with Code Evolution. arXiv:2602.16805

Sharma (2025)

OpenEvolve — github.com/algorithmicsuperintelligence/openevolve

Lange et al. (2025) · Assumpção et al. (2025) · Khrulkov et al. (2025)

ShinkaEvolve · CodeEvolve (arXiv:2510.14150) · GigaEvo (arXiv:2511.17592)

Mouret & Clune (2015) · Tanese (1989)

MAP-Elites · island-model genetic algorithms

Google DeepMind (May 2026, Jul 2026)

AlphaEvolve one-year impact report; general availability on Google Cloud

Tao (Nov 2025)

terrytao.wordpress.com — the clearest short explanation of why the loop works

Extra

Does the machinery earn its keep?

The AlphaEvolve ablation: remove one component, rerun, compare.

■ No evolution

Sample independently, never build on past programs. Biggest single loss.

■ No context in prompts

Drop the scored inspirations. The model stops learning the landscape.

■ No meta-prompt evolution

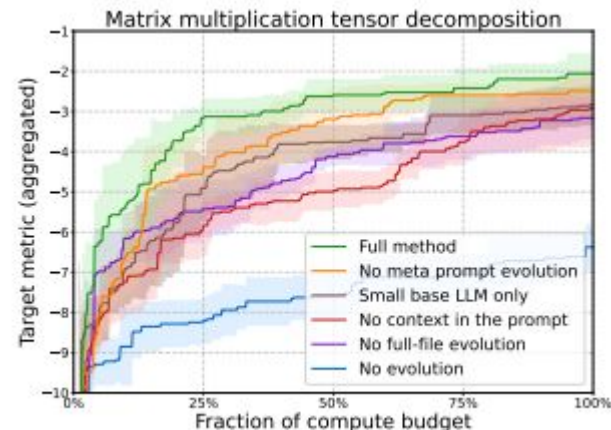
Smaller but consistent degradation.

■ No full-file evolution

Restrict to a single function, FunSearch-style. Hurts on the harder problems.

■ Small base model only

Cheap models suffice for easy problems; only frontier models solved the ones needing elaborate constructions.



The slide Google's blog post doesn't have

Simple Baselines are Competitive with Code Evolution

Yonatan Gideoni¹ Sebastian Risi² Yarin Gal¹

Google does not reveal the LLM budget they used for their studies and the prompts

Paper shows that designing the search space (evaluation) and the domain knowledge in the prompt set the performance ceiling.

reformulating one problem's search space improved the bound more than any pipeline difference, for every method at once. The reformulation was done by hand, using



Is your problem shaped like this?

1 Can a candidate be scored automatically?

Seconds to minutes, no human in the loop, no manual judgement.

2 Is the score honest?

Is there a cheap degenerate solution that scores well?

Assume it will be found.

3 Can you write down a search space?

Something a program can output.

This is the step that actually needs your expertise.

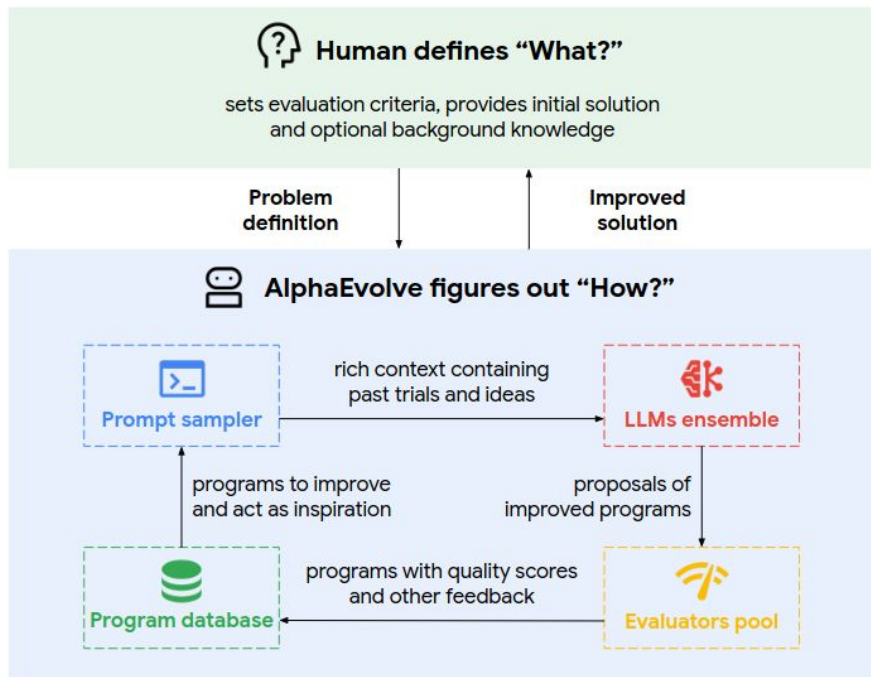
4 Is a few percent worth it?

These systems deliver small margins reliably. At scale that is money

5 Do you have a baseline and a held-out set?

Otherwise you cannot tell improvement from overfitting to your own evaluator.

The contract: you define "what", it figures out "how"



Read this as an interface, not an architecture

- You own the problem definition, the evaluation criteria and the initial solution.
- You may optionally hand over background knowledge — even raw PDFs of the relevant literature.
- What comes back is human-readable code you can inspect, debug and reject.



The evaluator is the only source of truth

■ The function h

A dictionary of scalar metrics, maximised by convention. Can be three lines returning a graph's size, or a full training run: FunSearch needed under 20 min on one CPU, AlphaEvolve can spend hours.

■ Evaluation cascade

Test cases of increasing difficulty; a candidate advances only if it did well at every earlier stage. Everything gets a tiny smoke-test first, to kill faulty programs cheaply.

■ LLM-generated feedback

For properties h can't express — simplicity of the program, say — a separate LLM call grades the candidate. Added to the score dict, or used as a hard filter.

■ Parallelised evaluation

~100 compute-hours per candidate is affordable, but a blocking evaluation starves the loop of generations. Most evaluation is embarrassingly parallel, so it goes async to a cluster.